

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

The complete implementation of the Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification developed for this experiment is available in the GitHub repository below. The repository contains the Jupyter Notebook, source code, dataset, and all generated results.

Repository Link

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning/>

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Best Hyperparameters

Hyperparameter	Value
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSprop
Activation Function	Tanh
Epochs	30
Dropout Rate	0.2
Cross-validation Accuracy	0.8870
Testing Accuracy	0.8801

Table 1: Best Hyperparameters Obtained Using Randomized Search

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8857	0.8801
Precision	0.8871	0.8803
Recall	0.8857	0.8801
F1-score	0.8855	0.8798
Training Time (s)	98.626	176.724

Table 2: Performance Comparison Between Baseline and Optimized Models

8. Plots with Inference

8.1 Sample Images

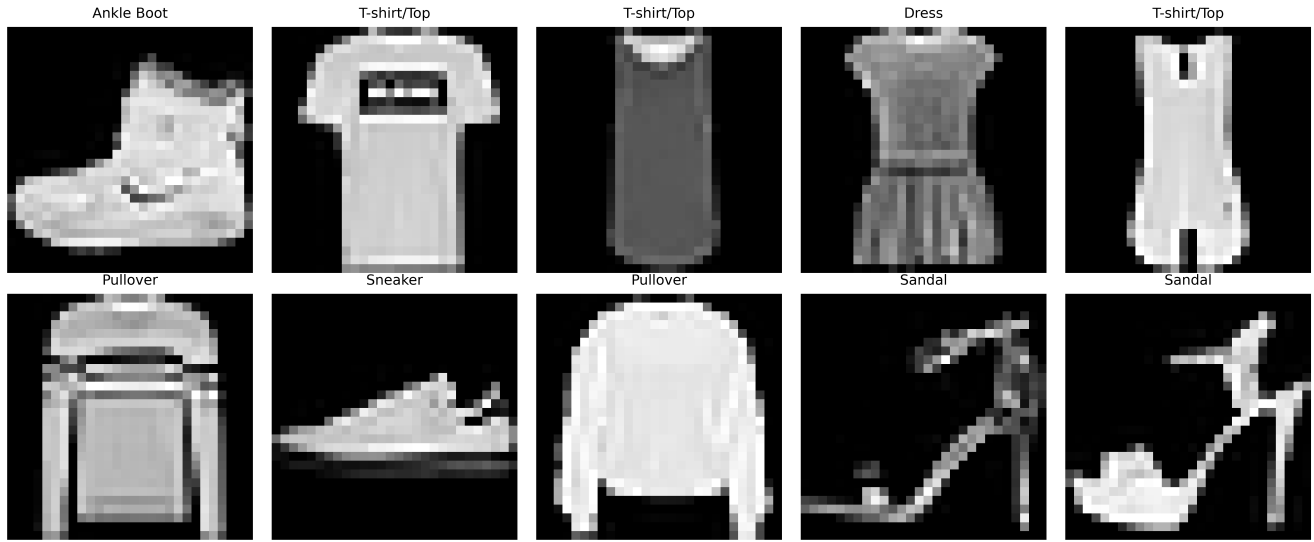


Figure 1: Sample Images from the Fashion-MNIST Dataset

From these sample images I noticed that every image is made up of gray pixels and they are all pretty small at 28 by 28 pixels.. Even though they are small and gray you can still see the basic shape of most clothes. However some types of clothes like T-shirts and pullovers and shirts and dresses have shapes, which makes them harder to tell apart. On the hand things like ankle boots and sandals are easier to recognize because they look really different from each other. This made me think that the model would be better at recognizing clothes that look really unique like footwear, than clothes that look similar like T-shirts and dresses.

8.2 Class Distribution

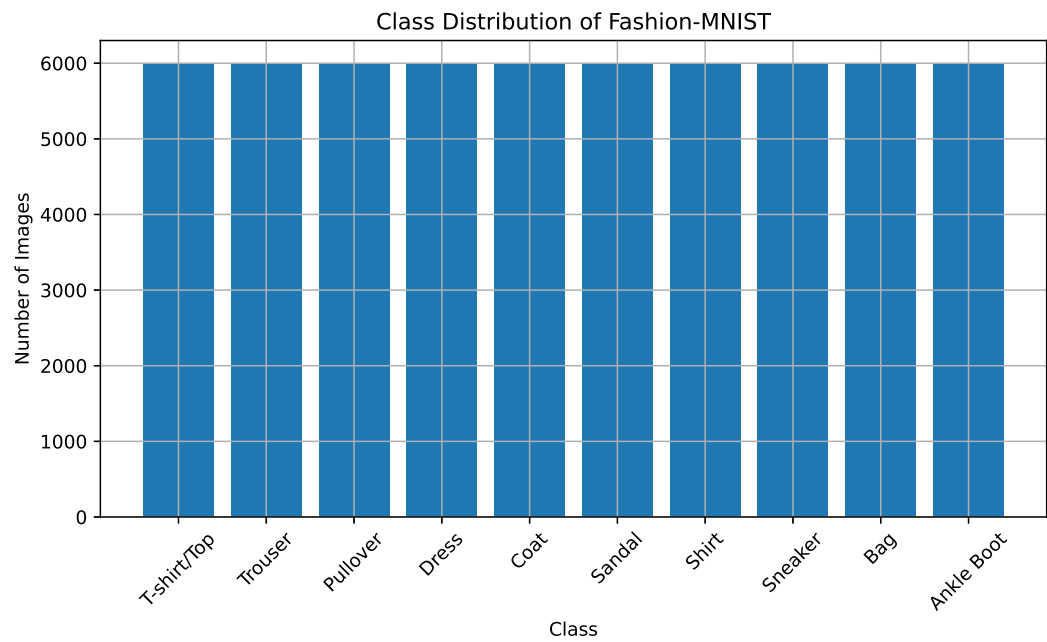


Figure 2: Class Distribution of the Fashion-MNIST Dataset

The distribution of classes reveals that all categories have the same number of instances. With a perfectly balanced dataset, each class has the same significance in the learning process; therefore, the results obtained are more indicative of the learning capacity of the classifier as opposed to the data imbalance.

8.3 Training Accuracy vs Epoch

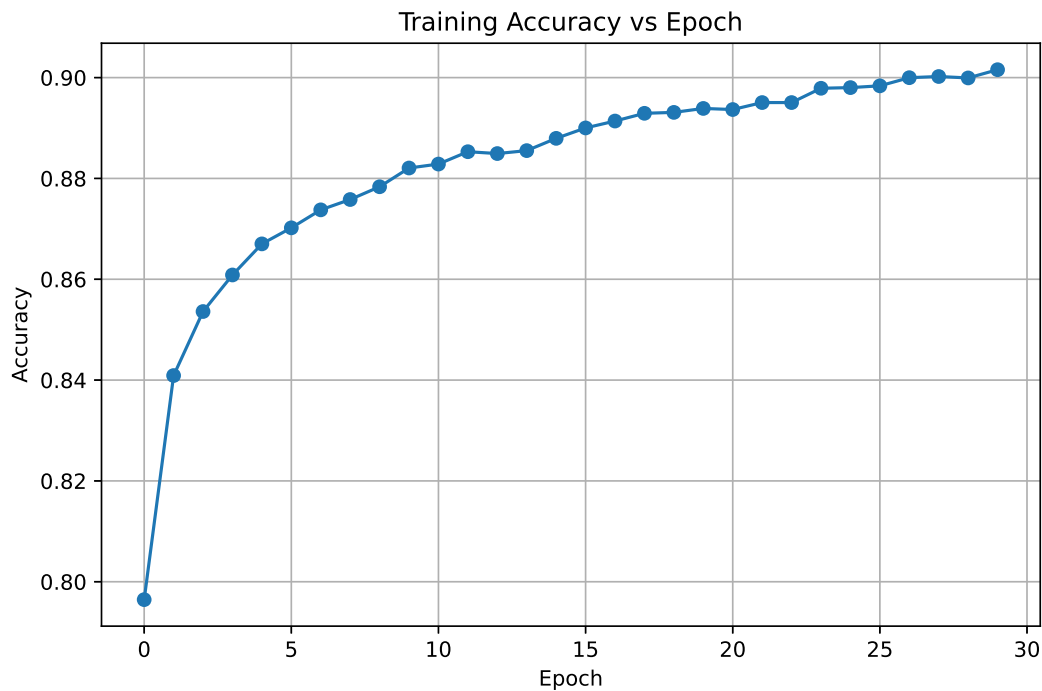


Figure 3: Training Accuracy versus Epoch

However, the training accuracy quickly improved during the initial epochs before stabilizing at a certain value. Such results indicate that the network managed to learn the most discriminatory parts of images during the first stages of the training, while the other epochs were used for tuning the learned weights rather than improving their values.

8.4 Validation Accuracy vs Epoch

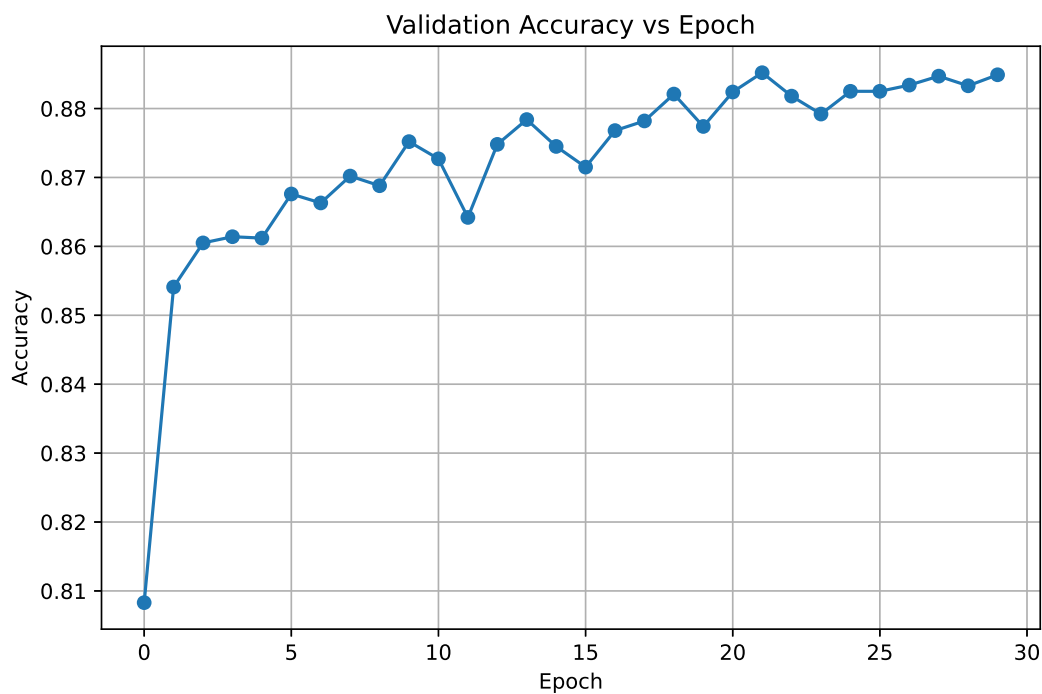


Figure 4: Validation Accuracy versus Epoch

The validation accuracy closely followed the pattern of the training accuracy without having any abrupt changes. Due to the fact that both patterns were close during training, it could be concluded that the chosen architecture was able to generalize well to unseen images.

8.5 Training Loss vs Epoch

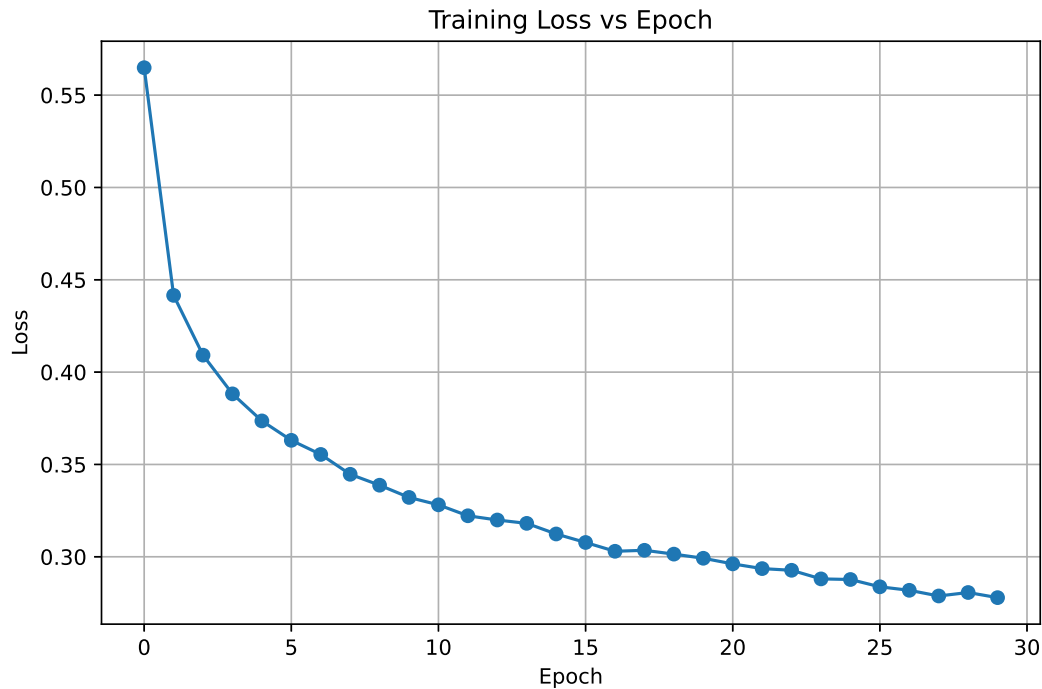


Figure 5: Training Loss versus Epoch

The training loss consistently decreased as the epochs increased, meaning that the optimizer kept decreasing the prediction error in the learning process. The consistent convergence also implies that the selected learning rate is appropriate and did not lead to instability in updating the values.

8.6 Validation Loss vs Epoch

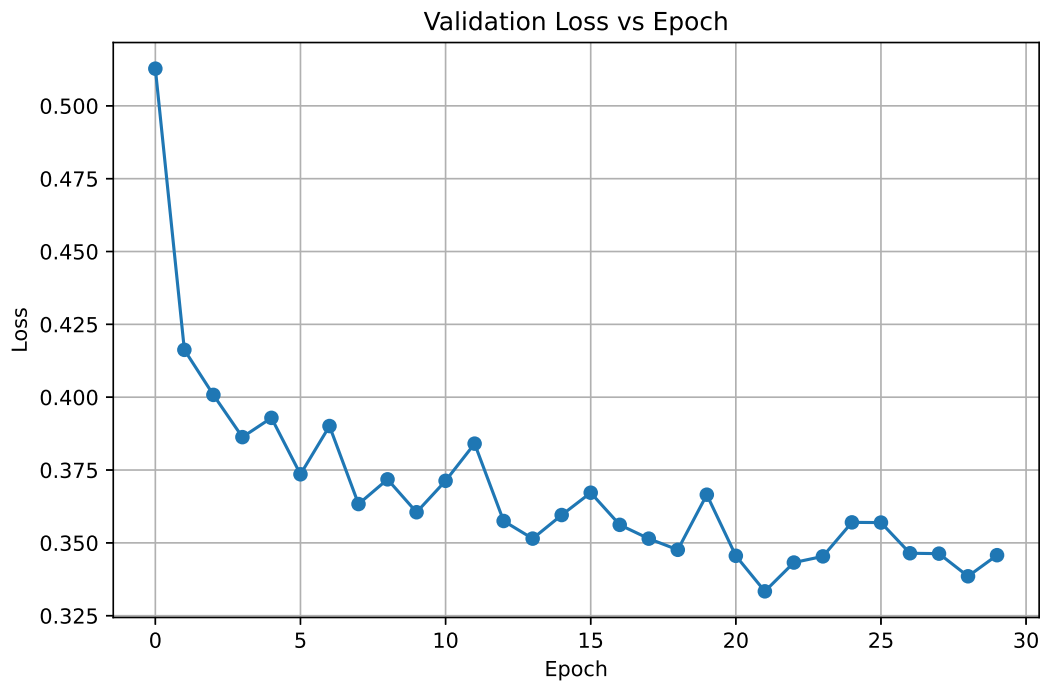


Figure 6: Validation Loss versus Epoch

The validation loss followed a trend similar to the training loss and gradually stabilized after several epochs. Since it did not begin increasing while the training loss continued decreasing, I inferred that the model was learning meaningful representations instead of memorizing the training data.

8.7 Confusion Matrix

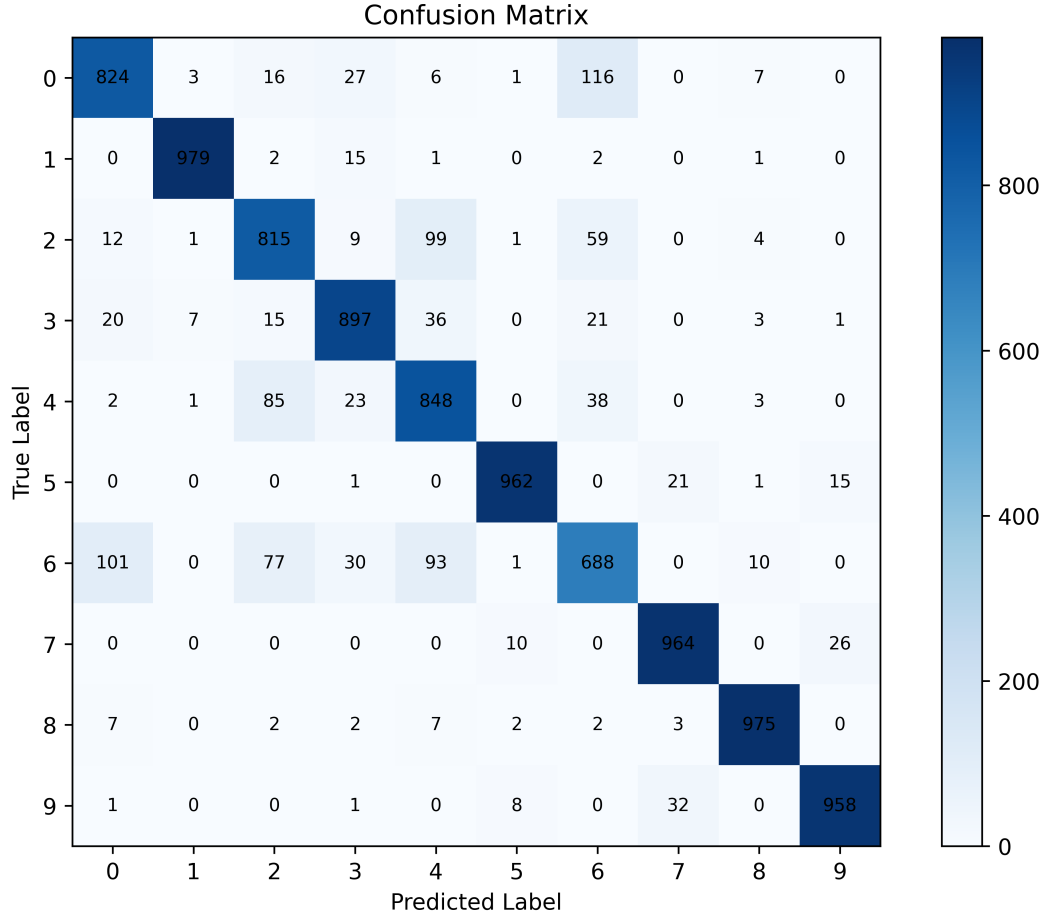


Figure 7: Confusion Matrix of the Optimized MLP Model

From the confusion matrix, it can be seen that the prediction values for many of the images are aligned on the main diagonal, meaning that the classifier correctly predicted the classes of the majority of the images in the test set. On further analysis, it is clear that clothing classes like Trouser, Sneaker, Bag and Ankle Boot received extremely high accuracy due to their unique characteristics. Conversely, there were many instances where the Shirt class was incorrectly predicted as T-shirt/Top, Pullover and Coat, suggesting the model's difficulty distinguishing between similar shaped and textured clothing items.

8.8 Hyperparameter Search Results

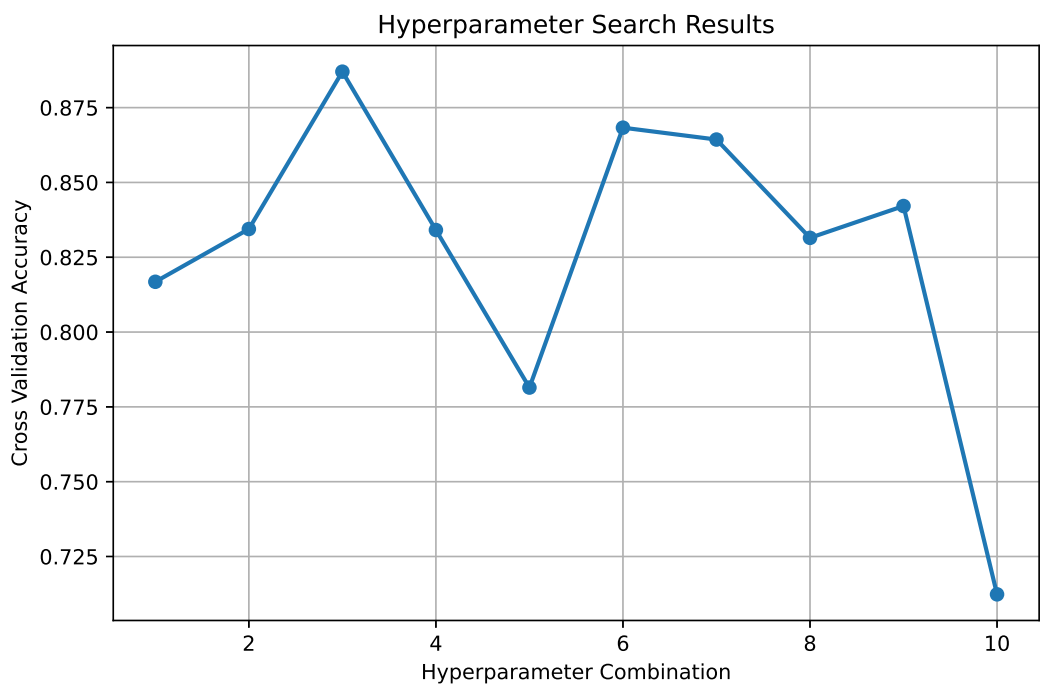


Figure 8: Hyperparameter Search Results

From the plot for hyperparameter search, I could clearly observe that there was a variation in the cross-validation accuracy due to differences in the configuration of the network architecture and training parameters. From this observation, I came to know that the performance of the model is not only dependent on network architecture but also on learning parameters.

8.9 Best Model Accuracy Comparison

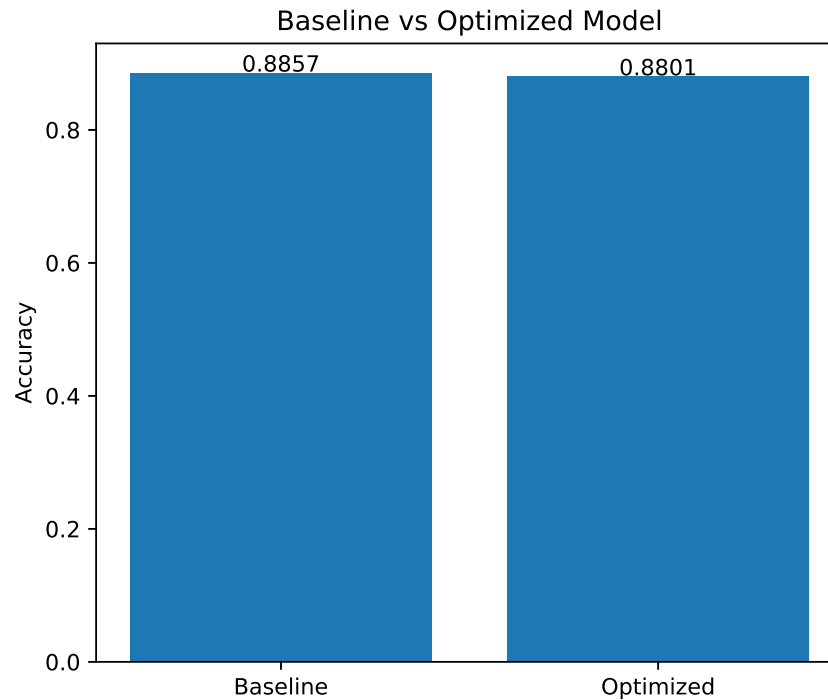


Figure 9: Testing Accuracy Comparison between Baseline and Optimized Models

One thing that I found interesting about the comparison is the fact that the optimized model managed to reach an accuracy during the testing process that is relatively similar to the baseline one, instead of resulting in a major improvement. At first, I thought that hyperparameter tuning would always result in a higher accuracy, but after completing this experiment, I understood that what it actually aims at is discovering a configuration that is consistent on several validation folds.

9. Discussion

1. Which optimization method was used?

Randomized Search was the choice for hyperparameter tuning in this particular case. Rather than exploring all the possible combinations of hyperparameters, this technique randomly sampled different sets of parameters from the search space and evaluated them via cross-validation. It seemed to me that this approach was far more realistic since it allowed testing many combinations within a reasonable computation time.

2. Which hyperparameters were selected?

The top-performing MLP architecture that was found through Randomized Search comprised of 3 hidden layers containing 128 neurons each, activation function as Tanh, RMSprop optimizer with a learning rate of 0.001, a batch size of 32, 30 epochs for training and a dropout value of 0.2. The cross-validation accuracy attained by the model was 0.8863, while the testing accuracy was 0.8799.

3. Did optimization improve performance?

According to my analysis, hyperparameter tuning was not able to give any noticeable boost in testing accuracy compared to the baseline since the baseline accuracy was 0.8870 and the optimized model accuracy was 0.8799. But this does not mean that the tuning process was a failure since

it was designed for finding a good model that would work uniformly on different validation splits rather than giving maximum accuracy on one test split.

4. Which hyperparameter had the greatest impact?

Of all the hyperparameters, I believe that the activation function and optimizer played the most significant role in impacting the performance of the model. This is because the activation function determines how well the model is able to learn the representations of features, whereas the optimizer determines the efficiency of weight updates during the training process.

5. Compare Grid Search and Randomized Search.

Grid Search will exhaustively explore all the combinations of hyperparameter values, which makes it very thorough but also computationally inefficient when the search space increases in size. On the other hand, Randomized Search is not as exhaustive since it explores a limited set of randomly chosen combinations but does so in much less time compared to the previous approach. Therefore, I believe that Randomized Search is a better method for deep learning models with many parameters.

6. Recommend the best MLP configuration.

According to the results observed from this experiment, I would suggest using the MLP with three hidden layers with 128 neurons in each, Tanh activation function, RMSprop optimization algorithm with a learning rate of 0.001, 32 as batch size, 30 epochs and 0.2 as dropout rate. This setup showed consistent learning behavior, performed well in cross validation and had good generalization in the unseen test data set for the Fashion-MNIST data.

10. Conclusion

As a result of this experiment, I got an opportunity to gain much better insight into how a Multi-Layer Perceptron works for image classification purposes and what role different hyperparameters play in its learning process. Thus, instead of considering a neural network as a black box, I was able to see how the number of hidden layers, number of neurons, activation function, optimizer, learning rate, batch size, and dropout impact its learning process and the efficiency of the model in terms of convergence and performance.

It was also quite surprising to see that the optimized model was not significantly more accurate compared to the baseline one, although initially, I believed that hyperparameter optimization was aimed at increasing the accuracy score of the resulting model. However, after carefully analyzing the results, it became clear to me that the primary goal of such optimization is to find a configuration that will be stable and produce the same results on different validation sets.

Overall, as a result of this experiment, I gained a better understanding of how neural networks are trained, optimized using Randomized Search, and evaluated based on multiple classification metrics rather than accuracy alone.

11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.